

An Improvement Faster Index for Property Matching Algorithm*

M. T. Juan¹, J. J. Liu^{2†} and Y. L. Wang¹

¹ Department of Information Management,
National Taiwan University of Science and Technology, Taipei, Taiwan
juan@mail.cute.edu.tw, ylwang@cs.ntust.edu.tw

² Department of Information Management,
Shih Hsin University, Taipei, Taiwan
jjliu@cc.shu.edu.tw

Abstract

Iliopoulos and Rahman proposed a data structure called the IDS-PIP for the property indexing pattern matching problem. The IDS-PIP can be constructed in $O(n)$ time. Then, based on ID-PIP, each query takes $O(m \log |\Sigma| + K)$ time, where n and m are the lengths of the text and the pattern, respectively, Σ is the alphabet and K is the output size. They assumed that all intervals in property π are disjoint. If the intervals in property π are not disjoint, then they create an equivalent set of disjoint intervals π' to replace π . However, in fact, property π' is not equivalent to property π at all. In this paper, we overcome this problem and modify IDS-PIP slightly so that Iliopoulos and Rahman's algorithm can work well again.

Keywords: Pattern matching, property pattern matching, suffix trees, property suffix trees.

1 Introduction

Pattern matching with properties is an interesting variant of the traditional pattern matching problem. In [2, 3], Amir et al. defined that the *property matching problem* is to find all occurrences of pattern P within some given ranges, called a *property*, of text T . If there are a set of patterns needed to

query, then, in general, the text has to be preprocessed so that each query can be done efficiently. In [3], Amir et al. gave an $O(n \log |\Sigma| + n \log \log n)$ time preprocessing algorithm to construct a data structure called a *property suffix tree* (PST for short), where n is the length of text, Σ is the alphabet, and $|\Sigma|$ is the cardinality of Σ . After PST is constructed, each query only takes $O(m \log |\Sigma| + K)$ time, where m is the length of the pattern and K is the number of occurrences of the pattern that satisfy the given property. Later in [6], Iliopoulos and Rahman proposed another data structure for solving this problem. Their algorithm takes only $O(n)$ preprocessing time for constructing a data structure called the *indexing data structure for property indexing problem* (IDS-PIP for short) if Σ is the set of natural numbers from 1 to a polynomial in n and $O(n \log \sigma)$ time otherwise where $\sigma = \min(n, |\Sigma|)$. The query time under IDS-PIP is the same as that of PST. However, there is an incorrect step in [6] on combining intervals in order to obtain an equivalent property. We shall explain it in the next section and give a solution for this problem without changing the time-complexity.

The remaining part of this paper is organized as follows. In Section 2, we introduce some definitions and notation used in this paper. Then, we point out the incorrect step in [6]. In Section 3, we propose a modified IDS-PIP for solving the property matching problem.

2 Preliminaries

In this paper, for easy of understanding the previous related paper, we use the same notation introduced in [6]. However, for ease of reading, we shall

*This work was supported in part by the National Science Council of the Republic of China under Contract NSC 96-2221-E-260-018.

†All correspondence should be addressed to Prof. J. J. Liu, 1 Lane 17 Sec. 1, Mu-Cha Rd. Taipei, Taiwan, R.O.C. (Email: jjliu@cc.shu.edu.tw).

introduce some of them. The reader is referred to [6] for terms undefined here. Given a text $\mathcal{T} = \mathcal{T}[1..n]$ of length n and a pattern $\mathcal{P} = \mathcal{P}[1..m]$ of length m , a traditional pattern matching problem is to find all occurrences i such that $\mathcal{P} = \mathcal{T}[i..i+m-1]$, for $1 \leq i \leq n-m+1$. We use $Occ_{\mathcal{T}}^{\mathcal{P}}$ to stand for the set of occurrences of $\mathcal{P}$ in $\mathcal{T}$. A *property* π of text $\mathcal{T}$ is a set of intervals $\pi = \{e_1, e_2, \dots, e_{|\pi|}\}$ and it holds that $1 \leq s_i \leq f_i \leq n$, for $1 \leq i \leq |\pi|$ where $e_i \equiv (s_i, f_i)$. An interval is *proper* if it is properly contained in some other interval. A property is called a *proper property* if there is no proper interval in it. A property π is said to be in *standard form* if it is a proper property and satisfies the condition $s_1 < s_2 < \dots < s_{|\pi|}$. Note that if the intervals in a property π are given in the ascending order of their left endpoints, then its corresponding proper property can be found in linear time. In the remaining part of this paper, we assume that all properties are in standard form. The *property indexing problem* (PIP problem for short) is to find all occurrences i such that $\mathcal{P} = \mathcal{T}[i \dots i+m-1]$ under the constraint that there exists an interval $(s_k, f_k) \in \pi$ containing $(i, i+m-1)$. The resulting occurrences of the PIP problem are denoted by $Occ_{\mathcal{T}, \pi}^{\mathcal{P}}$.

Two intervals e_i and e_j in property π are *overlapped* if $s_i < s_j < f_i$ for $1 < i < j < |\pi|$; otherwise, e_i and e_j are *disjoint*. A set of intervals is called a *disjoint interval set* if no two intervals in the set overlap each other. A set of consecutive intervals $e_i, e_{i+1}, \dots, e_j$ is called an *overlapped set* if $s_i < s_t < \max\{f_i, f_{i+1}, \dots, f_{t-1}\}$ holds for $t = i+1, i+2, \dots, j$. An overlapped set is *maximal* if it is not a proper subset of any other overlapped set. A maximal overlapped set is in a *decayed* form if there is only one interval in it. A nondecayed maximal overlapped set $\{e_i, e_{i+1}, \dots, e_j\}$ can be combined into a decayed maximal overlapped set by creating a new interval $e'_i = \{s_i, \max\{f_i, f_{i+1}, \dots, f_j\}\}$ to replace them. In [6], the authors assume that property π is disjoint before constructing IDS-PIP. If property π is not disjoint, then they decay every nondecayed maximal overlapped set to get the resulting set π' .

Unfortunately, $Occ_{\mathcal{T}, \pi'}^{\mathcal{P}}$ is not equivalent to $Occ_{\mathcal{T}, \pi}^{\mathcal{P}}$. We use an example to elucidate this fact. Assume that $\mathcal{T} = atacctac$, $\mathcal{P} = tac$, and $\pi = \{(2, 3), (3, 5), (6, 8)\}$. Property $\pi' = \{(2, 5), (6, 8)\}$ after π is decayed. We can find that $Occ_{\mathcal{T}, \pi'}^{\mathcal{P}} = \{2, 6\}$ while $Occ_{\mathcal{T}, \pi}^{\mathcal{P}} = \{6\}$.

3 A Modified IDS-PIP

In this section, we shall introduce the modified version of IDS-PIP (abbreviated as MIDS). Since this data structure is based on a suffix tree of text $\mathcal{T}$, we shall use the suffix tree in Figure 1 as an example on defining MIDS. For the construction of suffix trees, the reader is referred to [5, 11, 13] for the details. Note that, for consistency, the children of every node in the suffix tree shown in Figure 1 are arranged in an ascending order according to the labels on the edges between them and their parent node. Note also that the label, say i , beside every leaf node is the starting position of the suffix $\mathcal{T}[i..n]$ in $\mathcal{T}$. Actually, the labels of all leaf nodes constitutes a suffix array [1, 7, 8, 9, 10]. The first row of the MIDS is exactly the suffix array of text $\mathcal{T}$. We use $\mathcal{L}[i]$ to denote the first row of MIDS, namely the suffix array. The inverse function of $\mathcal{L}[i]$ is denoted by $\mathcal{L}^{-1}[j]$. Namely, if $\mathcal{L}[i] = j$, then $\mathcal{L}^{-1}[j] = i$. See the first row of Table 1 for an example. Note that the index row is not a part of the table.

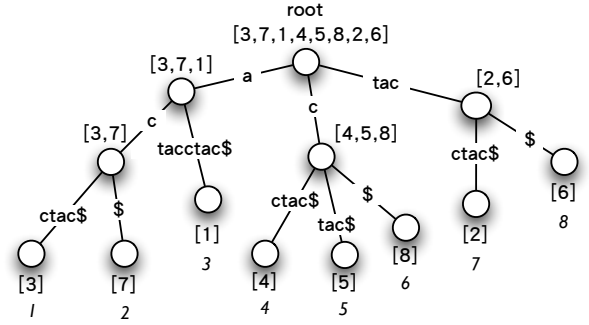


Figure 1: A suffix tree of $\mathcal{T} = atacctac$.

Table 1: The MIDS of $\mathcal{T} = atacctac$ and $\pi = \{(2, 3), (3, 5), (6, 8)\}$

i	1	2	3	4	5	6	7	8
$\mathcal{L}[i]$	3	7	1	4	5	8	2	6
$\mathcal{A}[i]$	3	2	-1	2	1	1	2	3

The values in the second row of MIDS are used to determine whether an occurrence in $Occ_{\mathcal{T}}^{\mathcal{P}}$ is also an occurrence in $Occ_{\mathcal{T}, \pi}^{\mathcal{P}}$. Assume that occurrence i is in $Occ_{\mathcal{T}}^{\mathcal{P}}$. Intuitively, to determine whether i is in $Occ_{\mathcal{T}, \pi}^{\mathcal{P}}$, we have to find an interval e_k from π which properly contains occurrence i . This can be achieved by checking whether $s_k < i$ and $f_k - i + 1 \geq m$. Or, conversely, for each position covered by

interval e_k , say i , we can precompute the maximum allowable length $f_k - i + 1$, denoted by $\mathcal{A}[i]$, for all possible occurrences i . If there is an occurrence i with length m , then we can simply check $\mathcal{A}[i] \geq m$ to determine whether occurrence i is in $Occ_{\mathcal{T}, \pi}^{\mathcal{P}}$ or not. Now, we introduce how to compute $\mathcal{A}[i]$ for $i = 1, 2, \dots, n$ by scanning the intervals in property π just once.

Algorithm 1: Algorithm for finding maximum allowable lengths.

Input: The length, say n , of a text and property $\pi = \{e_1, e_2, \dots, e_{|\pi|}\}$.
Output: Maximum allowable lengths $\mathcal{A}[i]$, $i = 1, 2, \dots, n$.
begin
Step 1. For each e_i , $1 \leq i \leq |\pi|$, compute $\mathcal{A}[\mathcal{L}^{-1}(k)] = f_i - k + 1$ for $s_i \leq k < s_{i+1}$, where we assume that $s_{|\pi|+1} = f_{|\pi|} + 1$.
Step 2. For $i = 1, 2, \dots, s_1 - 1$, set $\mathcal{A}[\mathcal{L}^{-1}(i)] = -1$.
Step 3. For $i = 1, 2, \dots, |\pi| - 1$,
 if $f_i + 1 < s_{i+1}$, then set $\mathcal{A}[\mathcal{L}^{-1}(k)] = -1$ for $f_i + 1 \leq k < s_{i+1}$.
Step 4. For $i = f_{|\pi|} + 1, f_{|\pi|} + 2, \dots, n$, set $\mathcal{A}[\mathcal{L}^{-1}(i)] = -1$.
End of Algorithm

We use an example to illustrate Algorithm 1. In Step 1, $\mathcal{A}[7], \mathcal{A}[1], \mathcal{A}[4], \mathcal{A}[5], \mathcal{A}[8], \mathcal{A}[2]$, and $\mathcal{A}[6]$ are computed since $\mathcal{L}^{-1}(2) = 7, \mathcal{L}^{-1}(3) = 1, \mathcal{L}^{-1}(4) = 4, \mathcal{L}^{-1}(5) = 5, \mathcal{L}^{-1}(6) = 8, \mathcal{L}^{-1}(7) = 2$, and $\mathcal{L}^{-1}(8) = 6$. Their corresponding values are $3 - 2 + 1 = 2, 5 - 3 + 1 = 3, 5 - 4 + 1 = 2, 5 - 5 + 1 = 1, 8 - 6 + 1 = 3, 8 - 7 + 1 = 2$, and $8 - 8 + 1 = 1$, respectively. In Step 2, $\mathcal{A}[\mathcal{L}^{-1}(1)] = \mathcal{A}[3] = -1$. In this example, no $\mathcal{A}[i]$ is computed from Steps 3 and 4. The second row of the MIDS in Table 1 contains all of the maximal allowable lengths for all positions in $\mathcal{T}$. Note that the relations of $\succ$, $\prec$, and $\doteq$ on $\mathcal{A}[i]$ are based on the lexicographic order of the pair $(\mathcal{A}[i], i)$.

After the data structured MIDS is constructed, the remaining procedures are similar to the algorithms described in [6]. For completeness, we also describe them as below. The notations used in these two algorithms are described in the following. The suffix tree of text $\mathcal{T}$ is denoted by $ST_{\mathcal{T}}$. The subtree of $ST_{\mathcal{T}}$ rooted at node v is denoted by $ST_{\mathcal{T}_v}$. Let $LL(v)$ denote the list (in left-to-right order) of the indices of the leaves of $ST_{\mathcal{T}_v}$. The leftmost (respectively, rightmost) leaf of subtree $ST_{\mathcal{T}_v}$ is de-

noted by $LML(ST_{\mathcal{T}_v})$ (respectively, $RML(ST_{\mathcal{T}_v})$). Every node v in $ST_{\mathcal{T}}$ has two pointers $v.left$ and $v.right$ which indicate the indexes of $LML(ST_{\mathcal{T}_v})$ and $RML(ST_{\mathcal{T}_v})$, respectively, in $LL(R)$, where R is the root of $ST_{\mathcal{T}}$. The substring obtained from the root of $ST_{\mathcal{T}}$ to node v is denoted by ℓ_v . The locus of pattern $\mathcal{P}$, denoted by $\mu^{\mathcal{P}}$, in $ST_{\mathcal{T}}$ is the node with which $\mathcal{P}$ is a prefix of $\ell_{\mu^{\mathcal{P}}}$ and $|\ell_{\mu^{\mathcal{P}}}|$ is the smallest of all such nodes.

Algorithm 2: Algorithm for query processing.

Input: Pattern $\mathcal{P}$.
Output: $Occ_{\mathcal{T}, \pi}^{\mathcal{P}}$.
begin
Step 1. Find locus $\mu^{\mathcal{P}}$ in suffix tree $ST_{\mathcal{T}}$
Step 2. Set $i = \mu^{\mathcal{P}}.left, j = \mu^{\mathcal{P}}.right$
Step 3. $Occ_{\mathcal{T}, \pi}^{\mathcal{P}} = \epsilon$.
Step 4. FindOccurrence($\mathcal{L}, i, j, \mathcal{P}$)
End of Algorithm

Algorithm 3: Procedure FindOccurrence($\mathcal{L}, i, j, m$).

Step 1. $h = \text{RangeMaximaQuery}(\mathcal{A}, i, j)[4, 12]$.
Step 2. If $\mathcal{A}[h] \geq m$
 Set $Occ_{\mathcal{T}, \pi}^{\mathcal{P}} = Occ_{\mathcal{T}, \pi}^{\mathcal{P}} \cup \mathcal{L}[h]$
 FindOccurrence($\mathcal{L}, i, h - 1, m$)
 FindOccurrence($\mathcal{L}, h + 1, j, m$)

End of Algorithm

References

- [1] M. I. Abouelhoda, E. Ohlebusch, S. Kurtz, Optimal exact string matching based on suffix arrays, in: *Lecture Notes in Computer Science*, vol. 2476, 2002, pp. 31–43.
- [2] A. Amir, E. Chencinski, C. S. Iliopoulos, T. Kopelowitz, and H. Zhang, Property matching and weighted matching, in: *Lecture Note in Computer Science*, vol. 4009, 2006, pp. 188–199.
- [3] A. Amir, E. Chencinski, C. S. Iliopoulos, T. Kopelowitz, and H. Zhang, Property matching and weighted matching, *Theoretical Computer Science*, 2007, in press.

- [4] M. A. Bender and M. Farach-Colton, The lca problem revisited, in: *Latin American theoretical INformatics (LATIN)*, 2000, pp. 88–94.
- [5] M. Farach, Optimal suffix tree construction with large alphabets, *Proceedings of the 38th Annual Symposium on Foundations of Computer Science*, 1997, pp. 137–143.
- [6] C. S. Iliopoulos and M. S. Rahman, Faster index for property matching, *Information Processing Letters*, 105, 2008, pp. 218–223.
- [7] J. Kärkkäinen, P. Sanders, and S. Burkhardt, Simple linear work suffix array construction, *Journal of the ACM*, 53, 2006, pp. 918–936.
- [8] D. K. Kim, J. S. Sim, H. Park, and K. Park, Constructing suffix arrays in linear time, *Journal of Discrete Algorithms*, 3, 2005, pp. 126–142.
- [9] P. Ko and S. Aluru, Space efficient linear time construction of suffix arrays, *Journal of Discrete Algorithms*, 3, 2005, pp. 143–156.
- [10] U. Manber and E. W. Myers, Suffix arrays: A new method for on-line string searches, *SIAM on Computing*, 22, 1993, pp. 935–948.
- [11] E. M. McCreight, A space-economical suffix tree construction algorithm, *Journal of the ACM*, 23(2), 1976, pp. 262–272.
- [12] K. Sadakane, Succinct data structures for flexible text retrieval systems, *Journal of Discrete Algorithms*, 5(1), 2007, pp. 12–22.
- [13] E. Ukkonen, On-line construction of suffix trees, *Algorithmica*, 14(3), 1995, pp. 249–260.